

Flask Search Engine - Project Documentation

1. Introduction

This Flask project is a modular search engine using Blueprints. It integrates external APIs such as Wikipedia, OMDb (IMDb), and NewsAPI to fetch data related to user queries. It also stores recent search history using SQLite.

2. Packages Used

1. Flask: For building the web application and handling routes.
2. requests: To send HTTP requests to external APIs.
3. sqlite3: To interact with the SQLite database for storing search history.
4. os: To access environment variables securely for API keys.

3. API Integrations and Getting API Keys

1. Wikipedia API (No Key Required):

- Used to fetch a summary and thumbnail for a search term.
- Endpoint: https://en.wikipedia.org/api/rest_v1/page/summary/{query}

2. OMDb API (Key Required):

- Used to fetch movie/show details like title, year, plot, and poster.
- Endpoint: http://www.omdbapi.com/?t={query}&apikey={OMDB_API_KEY}
- To get a free API key:
 - a. Visit <https://www.omdbapi.com/apikey.aspx>

Flask Search Engine - Project Documentation

- b. Sign up with your email.
- c. Select the free plan and copy your API key.

3. NewsAPI (Key Required):

- Used to fetch recent news articles related to the query.
- Endpoint: `https://newsapi.org/v2/everything?q={query}&apiKey={NEWS_API_KEY}`
- To get a free API key:
 - a. Visit `https://newsapi.org/register`
 - b. Create an account.
 - c. Get your API key from the dashboard.

4. Environment Variable Setup

To securely use API keys in the project, set them as environment variables:

On Linux/Mac:

```
export OMDB_API_KEY=your_omdb_key
```

```
export NEWS_API_KEY=your_newsapi_key
```

On Windows (CMD):

```
set OMDB_API_KEY=your_omdb_key
```

```
set NEWS_API_KEY=your_newsapi_key
```

Or create a `.env` file and use `dotenv` to load them securely in your Flask app.

Flask Search Engine - Project Documentation

5. SQLite Database

- The search history is stored in a SQLite database named `search\_history.db`.

- Table: history

Columns:

- id (Primary Key, Auto Increment)
- query (Text)
- timestamp (Datetime, default is current timestamp)

6. Conclusion

This project demonstrates how to build a modular and functional search engine with Flask, external API integration, and persistent search history using SQLite. It can be extended with features like user login, advanced filtering, and pagination.